Comprehension Questions: Function A1

Direction A: Identifying High-Level Stages

A1. What is the primary responsibility of the three consecutive early returns at the beginning of the function?

A. Convert ordering fields to a unified format.

B. Handle obviously invalid cases or cases that require no further checks before field-level validation.

C. Collect all valid fields in the model.

D. Check whether a related model exists.

A2. Which option best describes how fields are processed before related-field validation?

A. Remove all non-string fields, remove "?", and strip the leading "-" from field names.

B. Remove all fields containing LOOKUP_SEP and preserve the original ordering direction.

C. Convert all fields into model-field objects.

D. Keep only fields beginning with "-".

A3. Why does the code split fields into two lists, _fields and related_fields?

A. The two types of fields require different data structures.

B. Related fields must be parsed segment by segment along a path, whereas ordinary fields can be compared directly with the set of valid fields.

C. Ordinary fields can only be used for ascending order.

D. Related fields are not allowed in ordering.

Direction B: Stage Transitions and Control Flow

B1. What happens if cls._meta.order_with_respect_to is truthy?

A. Return models.E021.

B. Continue checking whether ordering is a list.

C. Return an empty error list immediately.

D. Check only related fields.

B2. When processing related_fields, what does the code do if the current path segment corresponds to a relationship field?

A. Add the field directly to errors.

B. Update _cls to the related model so that subsequent path segments can be resolved.

C. Remove the field and exit the loop.

D. Add the field to the set of ordinary fields.

B3. How does the ordinary-field validation stage identify invalid fields?

A. Compute valid_fields - fields.

B. Check whether a field contains LOOKUP_SEP.

C. Compute fields - valid_fields.

D. Check only whether the field equals "pk".

Direction C: Local Evidence and Module Boundaries

C1. Which location is the most suitable boundary between “field preprocessing” and “segment-by-segment validation of related fields”?

A. errors = []

B. fields = cls._meta.ordering

C. for field in related_fields:

D. return errors

C2. Which code fragment best represents the responsibility of parsing a relationship path segment by segment?

A. fields = (f.removeprefix('-') for f in fields)

B. for part in field.split(LOOKUP_SEP): ... _cls = fld.path_infos[-1].to_opts.model

C. invalid_fields.extend(fields - valid_fields)

D. return errors

C3. Which code fragment best represents collecting valid ordinary field names and identifying missing fields?

A. valid_fields = set(chain.from_iterable(...)) and invalid_fields.extend(fields - valid_fields)

B. _cls = cls and fld = None

C. if part == 'pk'

D. if cls._meta._ordering_clash

Overall Question

T1. Which option best summarizes the function?

A. Convert ordering fields uniformly into database column names.

B. Validate the overall ordering configuration, including configuration conflicts, type, relationship paths, and the existence of ordinary fields.

C. Automatically generate ordering fields for a model.

D. Perform database migrations for model fields.